
Contaminated Ancestors: Seven Ways Our Own Evaluation Protocol Nearly Told Us the Wrong Thing*

Christopher Yu-Hao Huang
Heritage High School
chuang19543@gmail.com

Ryan Jaemin Ahn
Yongsan International School of Seoul
ryanjmahn1@gmail.com

Wenhao Lu
Millburn High School
wenhao1u2027@gmail.com

Abstract

We report a full-length audit of a single scientific-ML evaluation protocol, conducted on our own work. The task is detecting chimeric ancestral protein reconstructions: given a family whose history contains recombination, does a scoring model plus a permutation test flag the contaminated segment? The headline metric was detection rate. We show that on this problem *detection rate has essentially no construct validity*: at $n = 15$ conditions the test model fires 2/15 and the null control fires 2/15, because a correctly calibrated permutation test is *supposed* to fire near α . The measurement that does discriminate is localization (mean Jaccard 0.81 vs 0.08 among firings), and the two dissociate in at least three independent places in our data. We then document six further ways the protocol produced or nearly produced a wrong conclusion: a sign convention that inverted a metric (AUC 0.048 where the truth was 0.952); a metric whose null is 0.50 to 0.68 rather than 0.5, so that 0.5 is the wrong baseline; a two-to-three seed regime in which seed variance exceeded every treatment effect and a 0/6 control looked immaculate before a 2/12 rerun showed it firing at nominal rate; a confounded comparison in which the treatment covaried with the number of input sequences; a competitor benchmarked at an undisclosed structural disadvantage; and an internal summary artifact reporting a run whose data was never committed, which could not be regenerated because the aggregation script had silently drifted from the result schema. Separately, we recorded an accelerator backend corrupting tensors mid-run *without raising*, converting $p = 0.030$ into $p = 1.000$ under identical settings. We close with a transferable protocol checklist. Every failure here is generic; none required the biology.

1 Introduction

Benchmarks in scientific machine learning are usually not leaderboards. There is often one dataset, one lab, a handful of seeds, a metric invented for the paper, and no external replication. Under those conditions the standard leaderboard pathologies, contamination, reuse, selective disclosure, are replaced by a different and less-studied family: *the protocol measures something other than the construct it names*, and there is nobody outside the lab positioned to notice [Pineau et al., 2021, Gundersen and Kjensmo, 2018, Bouthillier et al., 2021].

*Research conducted as part of AIscent Labs, <https://aiscent-research.github.io>.

This paper is a case study in that family, conducted against our own work. We do not report the scientific finding as such (it is a negative result, summarised in §2); we report the *evaluation*, because the interesting content is how many distinct ways a protocol built in good faith produced numbers that were locally correct and globally misleading.

Contributions.

1. A worked demonstration that the natural headline metric for a significance-test-based detector, detection rate, can have near-zero discriminative validity, with a quantitative dissociation from the metric that does discriminate (§3).
2. Six further documented protocol failures, each with the number it produced and the number that replaced it (§4).
3. An evaluation-integrity incident in which an accelerator backend silently returned corrupted tensors, producing well-formed but meaningless statistics (§5).
4. A transferable checklist (§6), annotated with which items our own protocol violated.

2 The task and the protocol under audit

Task. Maximum-likelihood ancestral sequence reconstruction (ASR) infers the sequence at an internal node of a phylogeny, one alignment column at a time [Yang et al., 1995, Pupko et al., 2000]. It assumes each lineage has exactly one parent. Recombination violates that, and ASR does not fail loudly: it returns a *chimeric* ancestor, one stretch from one sub-history and the next from another, with high per-site posteriors, because each site is inferred marginally and each site is individually defensible. We measure this directly: mean maximum posterior **0.93** at **69%** site accuracy against a known true root. **The pathology is joint; every standard diagnostic is marginal.**

The same protocol without the biology. Readers outside this application can substitute the following and lose nothing. A *family* is a dataset. A *reconstruction* is a model output of length L carrying one prediction per position. *Contamination* is a contiguous interval of positions generated by a different process than the rest, which the pipeline was supposed to notice and did not. A *score* assigns each position a real number measuring how much better it fits one candidate generating process than the other. The *detector* smooths that score, finds the interval whose mean most differs from the remainder, and asks a permutation test whether an interval that extreme would arise if position order were scrambled. Because the contaminated interval is known, every run yields two measurements: a binary verdict (did the test reject?) and an interval overlap (did it reject in the *right place*?). Everything in §§3–4 is a statement about that object. The protein machinery sets what the score is and how hard the instance is; it does not enter the argument about what the protocol measures.

Protocol. Reconstruct the two candidate sub-histories separately; score each site of the mosaic in each context; scan for a contiguous segment where the two contexts disagree; assess significance with a 200-permutation test that destroys spatial contiguity while preserving the marginal distribution of the per-site score, at $\alpha = 0.05$. Two scoring arms: *mpnn*, an inverse-folding network whose autoregressive decoder is a joint-compatibility model over residues on a backbone [Dauparas et al., 2022]; and *identity*, a two-term string-equality statistic using no structure and no network. A third arm keeps *mpnn* but permutes backbone coordinates, destroying the structural neighbourhood while preserving model, alphabet and composition.

Data generator and control. Families are simulated on one shared backbone. *selection* (test) evolves sequences by Gibbs sampling from the network’s joint distribution [Starr and Thornton, 2016]; *f81* (null control) evolves them site-independently toward per-site equilibrium profiles [Felsenstein, 1981]. Ground truth, the contaminated block, is known exactly, so both detection and localization are measurable. **This is a near-ideal evaluation setting:** synthetic, fully specified, with a matched negative control. That is the point of the paper. All seven failures below occurred anyway.

The scientific bottom line, for context. At 20 seeds, *identity* fires on 36/40 contaminated families (90%, Wilson CI [77%, 96%]) against *mpnn*’s 11/40 (28%, [16%, 43%]), which do not overlap. The structural model loses to string comparison. The backbone-scramble arm fires 2/40, so the *mpnn* signal that exists is genuinely structural, just weaker.

Table 1: Detection rate is identical across test and control; window accuracy is not. Note the non-independence: both control firings come from the same seed.

model	run	p	flagged window	true block	Jaccard
selection	seed 0, w30	0.020	50–92	55–85	0.71
selection	seed 1, w50	0.005	53–102	55–105	0.90
f81 (null)	seed 2, w10	0.015	39–103	55–65	0.16
f81 (null)	seed 2, w20	0.015	9–39	55–75	0.00
selection 2/12 fired				mean Jaccard	0.81
f81 2/12 fired				mean Jaccard	0.08

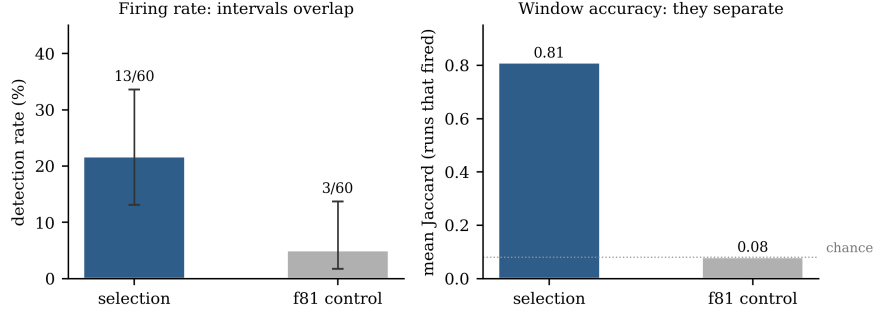


Figure 1: The construct-validity failure in one picture. Left: pooled detection rate at 20 seeds with 95% Wilson intervals; the test and null generators are not distinguishable, because a calibrated permutation test is supposed to fire near α . Right: mean localization Jaccard among the runs that fired. The headline metric does not separate them; the metric we were not reporting does.

3 Failure 1: the headline metric does not measure the construct

What we intended to measure. “Does the method detect contamination?” operationalised as the fraction of contaminated families on which the permutation test returns $p < \alpha$.

What it measured. Pooling our main experiment with a 12-condition segment sweep and de-duplicating shared conditions, *selection* fires 2/15 and the *f81* null control fires 2/15. **The rates are identical.** This is not a bug in the test; it is the test working. A calibrated permutation test at $\alpha = 0.05$ is *supposed* to fire at about α on null data, and at $n = 12$ –15 the binomial noise around α is the same order as the entire effect we were trying to measure.

What does discriminate. Where the flagged window lands (Table 1). Among runs that fired, mean Jaccard against the true block is 0.81 for *selection* and 0.08 for the control.

The dissociation is not a one-off. It recurs in three independent places in our data, which is what makes it a protocol property rather than an anecdote:

- **The null control** (above): fires at nominal rate, points nowhere (Jaccard 0.16 and 0.00).
- **The winning arm at large block width.** At width 80, *identity* fires 18/20 but 15 of those 18 firings land at Jaccard < 0.1 (mean 0.160, against 0.396 for *mpnn*’s 5 firings, of which 3 are also near zero). At width 50 the two are tied (0.683 vs 0.678). So the same arm, same metric, different condition, produces detections that are statistically real and substantively empty.
- **Fragmented contamination.** Splitting a fixed 50-residue contamination budget into multiple runs, the single firing that survives (*mpnn*, 3 blocks, seed 2, $p = 0.040$) lands at Jaccard 0.03: ten sites found against three true.

Diagnosis. The protocol’s inferential target was “is this family contaminated?” but the statistic’s alternative hypothesis is “does the sign sequence have more segmental structure than a spatial permutation of itself?” Those are different constructs, and a null family can satisfy the second

by chance. **Detection rate is a valid measurement of the test’s calibration and an invalid measurement of the method’s capability.** A usable protocol must report the window and its support, and needs a *second, independent* criterion to separate a real firing from a calibrated false one, which our work does not supply, and which we now regard as the main open methodological problem rather than a detail.

4 Six more ways the protocol misled us

4.1 Failure 2: a sign convention that inverted a metric

The segment scan is sign-symmetric: the intruding block and its complement are both “the window whose mean differs most from the rest.” Something label-free must break the tie. Our original rule, *the intrusion is the rarer sign among diagnostic sites*, is wrong, because outside the intruding block the mosaic is genuinely intermediate between the two sub-histories, so the per-site score is ≈ 0 with essentially random sign and the “majority” is decided by noise.

condition	detected	AUC, minority rule	AUC, segment rule
seed 0, block 55–85	yes ($p = 0.02$)	0.048	0.952
seed 1, block 25–55	no	0.828	0.172
seed 2, block 55–85	no	0.239	0.239

On the one run where detection succeeded, the protocol reported **AUC 0.048**: the correct signal, perfectly inverted. Had we shipped a slightly worse detector, this row would have read ≈ 0.5 and never drawn attention. The replacement orients by the window the scan itself flagged. It uses no ground truth and is not circular: the window is the detector’s *estimate*, and when the estimate is wrong the AUC lands below 0.5 rather than being rescued, which is what keeps it falsifiable.

Transferable point. A metric whose sign depends on a convention that ground truth never touches will fail silently, and it will fail *most visibly on the runs where the method works*, because those are the runs with signal to invert.

4.2 Failure 3: a metric whose null is not where the name suggests

The replacement orientation rule introduces a subtler problem: it inflates AUC on null data, because it always orients toward *some* window. Measured on the f81 control at matched block lengths, oriented AUC returns **0.50–0.68**, not 0.50.

So 0.5, the number every reader will supply from the name “AUC”, is not the right baseline for this statistic; the f81 series is. We report our oriented AUCs (e.g. 0.952 on the exemplar family) only against that series. We raise it here because a reviewer cannot detect this class of problem: nothing in the metric’s name, definition, or reported value reveals that its null has moved. **Any metric assembled from a data-dependent selection step needs its null measured, not assumed.**

4.3 Failure 4: the low-seed regime, where a control looked immaculate

In the main experiment the null control looked perfect: 0/6 detections, 0/3 false positives, site AUC 0.498 ± 0.058 , mean $p = 0.73$. We wrote “the control is silent.” Across the 12-condition sweep it fires **2/12**, twice at $p = 0.015$: close to nominal α , which is exactly what a correctly calibrated test should do. **The clean control was a small-sample artifact**, and the honest statement is not “the control is silent” but “the control fires at about its nominal rate.”

The same regime hid a larger problem: **seed variance exceeded every treatment effect we were studying.** On a divergence $\times$ clade-size grid at 2 seeds per cell, overall detection was 2/12 and *both* detections were in seed 0; seed 1 was 0/6 throughout. Any dose–response curve we drew through that grid would have been a description of one seed. Re-running the divergence axis at 6 seeds gives 2/6, 2/6, 4/6 across increasing divergence, with Wilson intervals [Wilson, 1927] overlapping substantially: *suggestive, not established*, and specifically not monotonic even though the covariate it supposedly acts through ($21.5 \rightarrow 36.2 \rightarrow 44.7$ mean diagnostic sites) is.

Table 2: The same comparison at 6 conditions and at 40. Only the second supports an inferential claim.

arm	6 matched conditions	20 seeds $\times$ 2 widths	95% Wilson CI
mpnn	2/6 (33%)	11/40 (28%)	[16%, 43%]
identity	4/6 (67%)	36/40 (90%)	[77%, 96%]
scrambled	0/6 (0%)	2/40 (5%)	[1%, 17%]

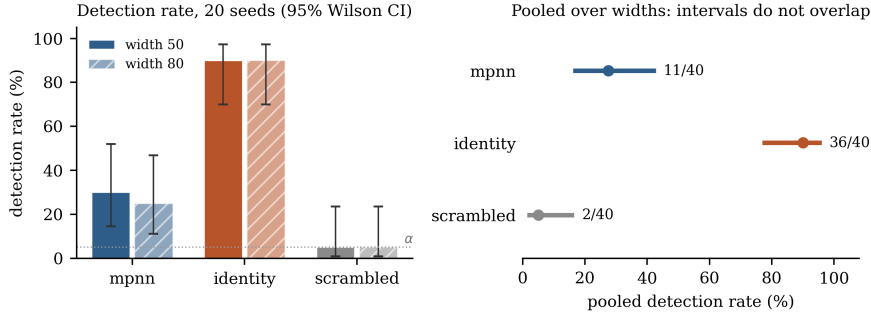


Figure 2: The same comparison at 20 seeds. At six conditions this was a 4/6-vs-2/6 suggestion; at forty it is a non-overlapping-interval result. The direction did not change, which is the point: at six conditions we could not have known that.

Re-running the ablation at 20 seeds is what turned the paper’s central claim from a 4/6-vs-2/6 suggestion into a non-overlapping-interval result (Table 2, Figure 2). The direction did not change. **The point is that at 6 conditions we could not have known that.**

We note a second, quieter statistical error the low-seed regime invited: *treating within-seed replicates as independent events*. Both null-control firings in Table 1 come from the same simulated family. Reported as “2/12” they look like two pieces of evidence; they are one.

4.4 Failure 5: a treatment confounded with sample size

We predicted that a chimeric ancestor should score worse under the joint model than a coherent sub-ancestor, with the gap widening with contamination, a clean directional prediction with an obvious metric. As originally measured, the mosaic was reconstructed from 12 input sequences and each sub-ancestor from 6. So “coherent” and “reconstructed from fewer sequences” varied together, and a reconstruction from more data sits closer to family consensus, exactly where a joint model assigns high likelihood.

comparison	mean gap	predicted sign
mosaic ($n=12$) vs sub-ancestor ($n=6$), confounded	$+0.025 \pm 0.033$	7/12
mixed ($n=6$) vs sub-ancestor ($n=6$), controlled	$+0.007 \pm 0.059$	4/12

The tell was available before the controlled experiment: the confounded gap is just as positive at *zero* contamination ($+0.036$). A directional prediction that is equally satisfied by the negative-control condition is not evidence for the mechanism. Controlled, the effect is indistinguishable from zero with the predicted sign in fewer than half of runs; on a real protein family the gap is *negative* (-0.23).

4.5 Failure 6: a competitor benchmarked at an undisclosed disadvantage

Our head-to-head against GARD [Kosakovsky Pond et al., 2006, 2020], a standard sequence-based recombination detector, shows GARD firing 0/6 on the test families where our arms fire 2/6 and 4/6 on identical data. Reported without qualification this reads as a clean win. It is not, for three reasons we now state inline with the result: (i) GARD is run in amino-acid mode because our setting has no nucleotide or codon layer to hand it, and codon-level analysis is where its power normally comes from; (ii) the alignments are 12 sequences over 106 sites, small for a partition-fitting method;

(iii) the test families are independently the *hardest* case by informative-site count (23–41 diagnostic sites versus 67–78 in the null families), so part of what GARD fails on is a floor that constrains our detector too.

The comparison that survives disclosure is qualitative and narrower: GARD *does* fire on the null families (3/6) but only ever finds *one* breakpoint, never the two that would bound a block, so it flags non-tree-likeness without localizing. And on our real 56-sequence, 58-site family GARD declines to run at all: HyPhy’s own assertion is that c-AIC model comparison needs at least 235 sites at that sequence count. **“Method A beats method B” was the wrong claim; “method A is computable in a regime where method B’s own parameter-count requirements rule it out” is the defensible one**, and it is a much weaker statement than our first draft made.

4.6 Failure 7: an internal artifact reporting a run that does not exist

The most alarming finding of the audit is not statistical. An internal summary document in our repository reported a substantially larger experiment: 12 seeds per condition on one axis, and a results table containing a protein family that does not appear in the corresponding audit output at all, its p -value traceable to a different result file. The committed data for that axis holds **five records**. The document could not be regenerated to check it: the aggregation script raises `KeyError` on the first section because it reads a schema the result files no longer use (records key on the number of blocks, not on a condition label), with a second latent schema break in a later section that expects a dictionary where the file holds a bare list.

Nothing in the analysis pipeline flagged either drift. The summary was human-readable, internally consistent, and wrong; and because the regeneration path was broken, its numbers had become unfalsifiable artifacts of a state of the code that no longer existed. **We treat this as the central lifecycle finding of the audit**: a results artifact that cannot be regenerated from committed data should be treated as retracted, not as evidence. We deleted it. In the same pass we found three committed figures rendered before the 20-seed data existed, still globbing result files that now resolve to the newer runs, and an aborted five-record sweep that no analysis cited but that sat in the results directory looking like data.

5 An evaluation-integrity incident: silent numerical corruption

Running long scoring sweeps on Apple silicon, we observed PyTorch’s MPS backend fail a Metal command buffer partway through a run and then *continue with corrupted tensors without raising an exception*. In a recorded incident, the same seed and the same settings that had produced $p = 0.030$ returned $p = 1.000$, with every score degraded to noise.

This is worth stating in an evaluation venue because of its shape: the failure produced **well-formed statistics in the expected range**. Nothing about $p = 1.000$ is structurally anomalous, and had the corruption occurred on a condition without a known answer we would have recorded it as a negative result. Our scorer now asserts finiteness of model output and raises otherwise, and every number reported in our work was produced on CPU. **Hardware-level silent corruption belongs on the threat model for any evaluation whose outputs are plausible-looking scalars**, which is to say, most of them.

6 A protocol checklist

None of the failures above required the biology. We state them as questions to ask of an evaluation protocol before running it at scale, and we mark with † the items our own protocol violated, which is nine of eleven. Several are standard advice. We report them anyway because **being standard advice did not prevent the failure**: the useful question is not whether a team knows to check for a confound, but whether anything in the workflow forces the check before the number reaches a draft.

1. † **What is the alternative hypothesis of your headline statistic, and is it the construct you are naming?** If the test’s alternative is satisfiable by chance under your null generator, the firing rate measures calibration, not capability.

2. † **Do detection and localization dissociate?** Report both. If your method can be “right” with a Jaccard of 0.00, the verdict is not the result.
3. † **Where is the null of every metric you report?** Measure it under the negative control rather than reading it off the metric’s name. Any selection step inside a metric moves its null.
4. † **Does any metric depend on a convention that ground truth never touches?** Sign conventions, tie-breaks and orientation rules fail silently and fail worst on your best runs.
5. † **Does the negative control’s clean result survive more seeds?** A 0/6 control and a 2/12 control support opposite sentences. Report intervals [Wilson, 1927], and check whether seed identity explains more variance than your treatment.
6. † **Are your “independent” observations independent?** Within-family replicates reported as a fraction inflate apparent evidence.
7. † **Does anything covary with your treatment?** Run the negative-control condition of your directional prediction: if the effect is equally present at zero treatment, you have measured a confound.
8. † **Is your baseline at a disadvantage you have not disclosed?** State the asymmetry inline with the result, and weaken the claim to what survives it.
9. † **Can every reported number be regenerated from committed data today?** Run the regeneration. Schema drift is silent, and an artifact you cannot regenerate is not evidence.
10. **Would you notice numerical corruption?** Assert on model output; prefer the boring device.
11. **Ablate the model with a non-learned stand-in in the identical pipeline.** This was our single highest-value experiment: it cost minutes and it relocated the contribution from the model to an upstream step.

Item 11 deserves emphasis because it is the one that changed our scientific conclusion. Our probe is a composite: (1) reconstruct two sub-histories separately; (2) compare the candidate against each; (3) scan for a contiguous run where the comparison flips. We attributed the method’s power to a large model doing step (2). Substituting string equality, same scan, same permutation test, same data, so any difference is a difference in *signal*, not procedure, located it in step (1). **Evaluating the pipeline told us it worked; evaluating the component told us which component.**

Which of these are domain-general. Failures 1 through 5 involve no biology at all. They are properties of a detector scored against known ground truth with a permutation test, and any field running that shape of evaluation can reproduce every one of them with a different score function. Failure 6 is general in kind but domain-bound in its specifics, since what counts as a fair configuration for a competitor depends on the competitor. Failure 7 and the corruption incident of §5 are properties of research software rather than of any domain. What a single case cannot establish is base rates: we can show that these failures co-occur in one pipeline and that a near-ideal evaluation setting did not prevent them, not how often each occurs across pipelines. Measuring that would require auditing many protocols the way we audited one, which we think is the natural next study.

7 Limitations of this audit

This is a single protocol in a single domain, audited by its authors; we can enumerate the failures we found, not the ones we did not. The synthetic generator that makes ground truth available also makes this an unusually favourable evaluation setting, so the base rate of protocol failure in messier settings is plausibly higher, not lower. Several of our own corrected numbers still rest on 2–3 seeds: notably the spatial-scan interaction ($n = 9$ per cell) and the fragmentation sweep (3 seeds). We mark these as suggestive rather than established instead of re-running everything. The divergence grid remains underpowered after tripling. Our simulator draws from the same model family as one of the scoring arms, which is a circularity we control for with the independent null generator and with real data but do not eliminate. Two implementation bugs found during the audit are disclosed and fixed: a contamination-budget layout that floor-divided and dropped a remainder (48 residues allocated against a stated 50 in two conditions), and a results filename hardcoded to one family that caused one

run to overwrite another’s output, caught immediately from mismatched printed metadata, restored from version control, no reported result affected.

Reproducibility statement

All reported numbers were produced with `-device cpu` for the reason given in §5. Fixed configuration: one backbone (PDB 5L33 chain A, 106 residues), inverse-folding checkpoint `v_48_020`, sampling temperature 0.5, 8 sweeps per unit branch length, 200 permutations, $p = (1 + \#\{\text{null} \geq \text{obs}\}) / (1 + n_{\text{perm}})$, $\alpha = 0.05$, 12 sequences per family (6 per clade) unless stated. External tools: `mafft` [Kato and Standley, 2013], `iqtree3` [Minh et al., 2020], `hyphy 2.5.101` [Kosakovsky Pond et al., 2020], `colabfold_batch` [Mirdita et al., 2022]. Python 3.12, PyTorch, NumPy.

Use of large language models

An LLM was used for prose editing, LaTeX formatting and literature-search assistance. It did not design experiments, generate or analyse data, or produce any numerical result reported here; all numbers derive from committed result files and were checked against raw output during the audit described in §4. The authors take full responsibility for the content.

References

- Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, et al. Accounting for variance in machine learning benchmarks. *Proceedings of Machine Learning and Systems*, 3:747–769, 2021.
- Justas Dauparas, Ivan Anishchenko, Nathaniel Bennett, Hua Bai, Robert J. Ragotte, et al. Robust deep learning-based protein sequence design using ProteinMPNN. *Science*, 378(6615):49–56, 2022.
- Joseph Felsenstein. Evolutionary trees from DNA sequences: a maximum likelihood approach. *Journal of Molecular Evolution*, 17(6):368–376, 1981.
- Odd Erik Gundersen and Sigbjørn Kjensmo. State of the art: Reproducibility in artificial intelligence. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 2018.
- Kazutaka Katoh and Daron M. Standley. MAFFT multiple sequence alignment software version 7: improvements in performance and usability. *Molecular Biology and Evolution*, 30(4):772–780, 2013.
- Sergei L. Kosakovsky Pond, David Posada, Michael B. Gravenor, Christopher H. Woelk, and Simon D. W. Frost. Automated phylogenetic detection of recombination using a genetic algorithm. *Molecular Biology and Evolution*, 23(10):1891–1901, 2006.
- Sergei L. Kosakovsky Pond, Art F. Y. Poon, Roger Velazquez, Steven Weaver, et al. HyPhy 2.5—a customizable platform for evolutionary hypothesis testing using phylogenies. *Molecular Biology and Evolution*, 37(1):295–299, 2020.
- Bui Quang Minh, Heiko A. Schmidt, Olga Chernomor, Dominik Schrempf, Michael D. Woodhams, Arndt von Haeseler, and Robert Lanfear. IQ-TREE 2: New models and efficient methods for phylogenetic inference in the genomic era. *Molecular Biology and Evolution*, 37(5):1530–1534, 2020.
- Milot Mirdita, Konstantin Schütze, Yoshitaka Moriwaki, Lim Heo, Sergey Ovchinnikov, and Martin Steinegger. ColabFold: making protein folding accessible to all. *Nature Methods*, 19(6):679–682, 2022.
- Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, et al. Improving reproducibility in machine learning research. *Journal of Machine Learning Research*, 22(164): 1–20, 2021.
- Tal Pupko, Itsik Pe’er, Ron Shamir, and Dan Graur. A fast algorithm for joint reconstruction of ancestral amino acid sequences. *Molecular Biology and Evolution*, 17(6):890–896, 2000.

Tyler N. Starr and Joseph W. Thornton. Epistasis in protein evolution. *Protein Science*, 25(7): 1204–1218, 2016.

Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

Ziheng Yang, Sudhir Kumar, and Masatoshi Nei. A new method of inference of ancestral nucleotide and amino acid sequences. *Genetics*, 141(4):1641–1650, 1995.

A The governing covariate, and why we nearly reported the wrong one

Our first dose–response analysis used contaminated block length as the independent variable, which is the parameter we set. It is not the variable that governs the outcome.

block length	detections	informative (diagnostic) sites inside block, per seed
10	0/3	5, 1, 3
20	0/3	8, 0, 4
30	1/3	13 , 0, 2
50	1/3	22, 22 , 16

No run with fewer than 13 diagnostic sites inside the block ever fired (0 of 8); 2 of the 4 runs with ≥ 13 did. Two of the twelve blocks contained *zero* diagnostic sites and were undetectable in principle, meaning two cells of our design contributed guaranteed failures to a rate we were reporting as a capability measurement. The apparent effect of block length is largely this covariate wearing a costume: longer blocks tend to contain more diagnostic sites, but the correlation is loose enough that a 50-residue block with 16 sites failed at $p = 0.33$ while a 30-residue block with 13 succeeded at $p = 0.02$.

The same covariate explains an apparent second effect. In the divergence grid, both detections occurred at 6 sequences per clade and none at 3 (0/6 across all divergence levels), but the two clade sizes at matched divergence have the *same* mean diagnostic-site count (31.0 vs 30.0), so here the covariate is *not* the explanation and input count genuinely matters. Distinguishing those two cases required measuring the covariate per condition, which we did not do in the first analysis pass.



Figure 3: The protocol on its best case ($p = 0.0199$, Jaccard 0.714, oriented AUC 0.952, read against the 0.50–0.68 null of §4, not against 0.5). Included to make the object of the audit concrete; no claim in this paper depends on it.